Estructuras de Datos y Algoritmos

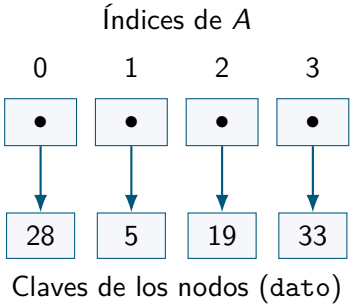
Tema 13: Tablas hash

Dr. Jorge Armando Ramírez Díaz

Universidad Nacional Autónoma de México
ENES Unidad Juriquilla
Licenciatura en Matemáticas para el Desarrollo

Buscar por clave

El arreglo A guarda referencias a nodos. Cada nodo contiene una clave en su atributo `dato` y puede guardar información adicional.



Consultar una posición

`A[2]` devuelve directamente la referencia al nodo cuyo `dato` es 19. Acceder a esa posición cuesta $\Theta(1)$.

Buscar una clave

Conocer la clave 19 no nos dice que el nodo está referenciado desde la posición 2.

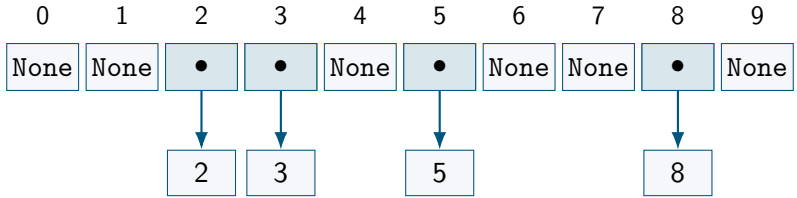
¿Podemos usar la clave del nodo para determinar en qué posición buscarlo?

Direccionamiento directo

Definición: tabla de direccionamiento directo

Un arreglo con una casilla por cada clave del universo $U = \{0, 1, \ldots, m-1\}$. La casilla $T[k]$ referencia al elemento con clave k, o contiene `None` si no está almacenado.

$$U = \{0, 1, \ldots, 9\}, \qquad K = \{2, 3, 5, 8\}.$$



Cada nodo puede guardar también datos asociados.

Aquí la **clave coincide con el índice** de la casilla.

Operaciones directas

Función BuscarDirecto(T, k)

Retornar $T[k]$

Procedimiento InsertarDirecto(T, nodo)

T[nodo.dato] $\leftarrow$ nodo

Procedimiento EliminarDirecto(T, nodo)

T[nodo.dato] $\leftarrow$ None

```
def buscar_directo(T, k):
    return T[k]

def insertar_directo(T, nodo):
    T[nodo.dato] = nodo

def eliminar_directo(T, nodo):
    T[nodo.dato] = None
```

Cada operación cuesta $\Theta(1)$. Crear `T = [None] * m` cuesta $\Theta(m)$, aunque almacenemos muy pocas claves.

Tabla hash

Definición: función de dispersión

Una función hash asigna a cada clave una casilla de la tabla:

$$h : U \longrightarrow \{0, 1, \ldots, m-1\}.$$

La tabla utiliza $h(k)$ para localizar dónde almacenar o buscar la clave k.



- **Clave:** identifica el elemento que buscamos.
- **Índice:** indica una casilla del arreglo.
- **Elemento:** contiene la clave y, eventualmente, otros datos.

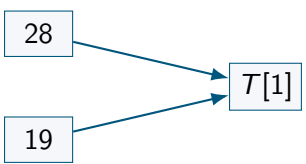
Podemos elegir m mucho menor que $|U|$.

Colisiones

Definición: colisión

Dos claves distintas colisionan cuando llegan a la misma casilla:

$$k_1 \neq k_2, \qquad h(k_1) = h(k_2).$$



Si $|U| > m$, hay más claves posibles que casillas. Por el principio del palomar, alguna pareja debe compartir casilla.

La colisión **no implica igualdad**.

Resolveremos las colisiones mediante **encadenamiento** y **direccionamiento abierto**.

Función de dispersión

Método de división

Para una clave entera $k \geq 0$ y una tabla de tamaño $m > 0$, definimos:

$$h(k) = k \bmod m, \qquad 0 \leq h(k) < m.$$

El residuo determina una posición válida de la tabla.

Ejemplo con $m = 9$

Clave k	5	28	19	15	20	33
k mód 9	5	1	1	6	2	6

$28 = 3 \cdot 9 + 1$ y $19 = 2 \cdot 9 + 1$: ambas claves llegan a la casilla 1.

Costo: calcular $h(k)$ requiere $\Theta(1)$.

- La función es determinista: la misma clave produce el mismo índice.
- Si las claves siguen un patrón, muchas pueden colisionar.

Multiplicación

Método de multiplicación

Para una clave entera $k \geq 0$, tamaño $m > 0$ y constante fija $0 < A < 1$:

$$h(k) = \lfloor m\,(kA \bmod 1) \rfloor\,, \qquad 0 \leq h(k) < m.$$

$kA \bmod 1 = kA - \lfloor kA \rfloor$ es la parte fraccionaria de kA.

Ejemplo: $m = 8$, $A = 13/32$ y $k = 19$.

$$kA = 19 \cdot \frac{13}{32} = \frac{247}{32} = 7 + \frac{23}{32}.$$

Tomamos la parte fraccionaria, la multiplicamos por m y aplicamos el piso:

$$h(19) = \left\lfloor 8 \cdot \frac{23}{32} \right\rfloor = \left\lfloor \frac{23}{4} \right\rfloor = 5.$$

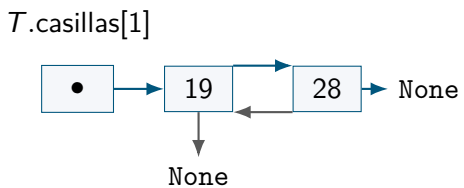
Podemos elegir m independientemente de A.
Costo: $\Theta(1)$.

Encadenamiento

Cada casilla referencia la cabeza de una lista con las claves que llegan a ella.

```
class NodoHash:
    def __init__(self, dato):
        self.dato = dato
        self.anterior = None
        self.siguiente = None


class TablaHash:
    def __init__(self, m):
        self.casillas = [None] * m
```



Azul: siguiente. Gris: anterior.

Una casilla sin elementos contiene `None`.

Las listas son **doblemente enlazadas**: podremos retirar en tiempo constante un nodo que ya tengamos localizado. La clave de un nodo no cambia mientras está en la tabla.

Insertar

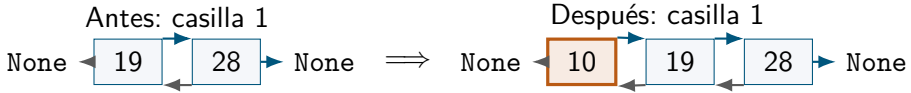
Precondición: nodo nuevo, clave ausente y tabla con $m > 0$ casillas.

Método InsertarEncadenado(nodo)

$m \leftarrow$ |self.casillas|
$j \leftarrow$ nodo.dato mód m
nodo.anterior $\leftarrow$ None
nodo.siguiente $\leftarrow$ self.casillas[j]
Si self.casillas[j] no es vacío:
    self.casillas[j].anterior $\leftarrow$ nodo
self.casillas[j] $\leftarrow$ nodo

```
def insertar_encadenado(self, nodo):
    m = len(self.casillas)
    j = nodo.dato % m
    nodo.anterior = None
    nodo.siguiente = self.casillas[j]
    if self.casillas[j] is not None:
        self.casillas[j].anterior = nodo
    self.casillas[j] = nodo
```

Método de `TablaHash`. Ejemplo: `T.insertar_encadenado(NodoHash(10))`, con $m = 9$.



Costo: $\Theta(1)$. El nuevo nodo queda al inicio, con `anterior = None`.

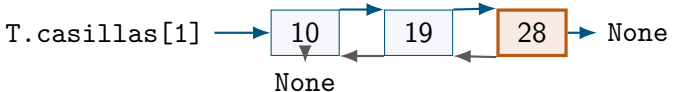
Buscar

Método BuscarEncadenado(k)

$m \leftarrow$ |self.casillas|
$j \leftarrow k$ mód m
nodo $\leftarrow$ self.casillas[j]
Mientras nodo no es vacío
y nodo.dato $\neq k$:
    nodo $\leftarrow$ nodo.siguiente
Retornar nodo

```
def buscar_encadenado(self, k):
    m = len(self.casillas)
    j = k % m
    nodo = self.casillas[j]
    while (nodo is not None
           and nodo.dato != k):
        nodo = nodo.siguiente
    return nodo
```

Ejemplo: `T.buscar_encadenado(28)`, con $m = 9$ y $j = 1$. Retorna el nodo resaltado.



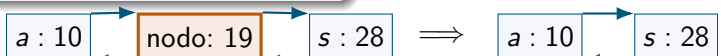
Costo: $\Theta(\ell)$, con ℓ nodos examinados y $0 \leq \ell \leq n$. En el peor caso se examinan los n elementos de la tabla: $\Theta(n)$.

Eliminar

Método EliminarEncadenado(nodo)

$m \leftarrow$ |self.casillas|
$j \leftarrow$ nodo.dato mód m
$a \leftarrow$ nodo.anterior
$s \leftarrow$ nodo.siguiente
Si a es vacío:
    self.casillas[j] $\leftarrow s$
Si no:
    a.siguiente $\leftarrow s$
Si s no es vacío:
    s.anterior $\leftarrow a$

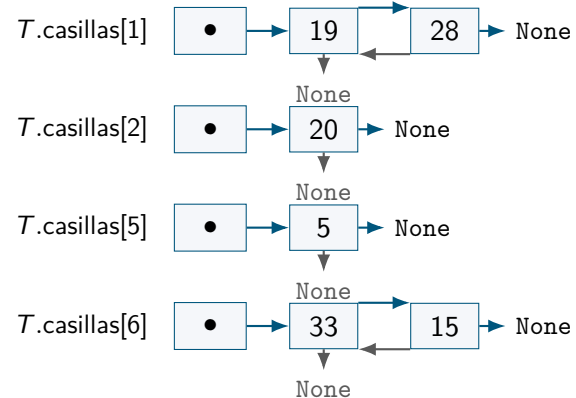
```
def eliminar_encadenado(self, nodo):
    j = nodo.dato % len(self.casillas)
    a = nodo.anterior
    s = nodo.siguiente
    if a is None:
        self.casillas[j] = s
    else:
        a.siguiente = s
    if s is not None:
        s.anterior = a
```



Costo: $\Theta(1)$. Si solo conocemos la clave, primero hay que buscar el nodo.

Seguimiento

Ejemplo independiente: creamos `T = TablaHash(9)`. Insertamos $5, 28, 19, 15, 20, 33$, en ese orden, mediante `T.insertar_encadenado(NodoHash(k))` para cada clave k.



Las casillas $0, 3, 4, 7, 8$ contienen `None`.

Cada operación parte del dibujo.

- `T.buscar_encadenado(19)` devuelve el nodo 19, cabeza de la casilla 1.
- `T.buscar_encadenado(46)` examina 19 y 28 y devuelve `None`.

Para retirar el nodo con dato 19:

```
nodo = T.buscar_encadenado(19)
T.eliminar_encadenado(nodo)
```

`T.casillas[1]` pasa a referenciar al nodo 28 y su atributo `anterior` queda en `None`.

Elegir la función

Una función fija puede distribuir mal ciertos conjuntos de claves. El **hashing universal** elige al azar una función de una familia adecuada.

Definición: familia universal

Una familia finita $\mathcal{H}$ es universal si, al elegir h uniformemente de ella, para cualesquiera claves distintas k_1, k_2,

$$\Pr_{h \in \mathcal{H}}[h(k_1) = h(k_2)] \leq \frac{1}{m}.$$

1. Elegir la función independientemente de las claves que se almacenarán.
2. Conservar esa función para insertar, buscar y eliminar.

La aleatoriedad está en **elegir la función**, no en cambiar el índice de una clave cada vez que la consultamos.

Una familia universal

Sea p un primo, $U = \{0, \ldots, p-1\}$ el universo de claves y $1 \leq m < p$ el tamaño de la tabla.

Familia universal

$$h_{a,b}(k) = ((ak + b) \bmod p) \bmod m,$$

con $a \in \{1, \ldots, p-1\}$ y $b \in \{0, \ldots, p-1\}$.

Cada pareja (a, b) determina una función de la familia.

Elegimos a y b uniformemente e independientemente entre sí y de las claves, **una sola vez**.

Ejemplo: $p = 17,\ m = 6,\ a = 3,\ b = 4.$

$$\begin{aligned} h_{3,4}(8) &= ((3 \cdot 8 + 4) \bmod 17) \bmod 6 \\ &= (28 \bmod 17) \bmod 6 \\ &= 11 \bmod 6 = 5. \end{aligned}$$

Los parámetros a y b quedan fijos.
La garantía probabilística corresponde a la elección de la función. Una función elegida todavía puede producir colisiones.

Direccionamiento abierto

Cada casilla de `self.casillas` almacena directamente una clave o `None`.

Definición: secuencia de sondeos

Para una clave fija k, i indica el intento de examinar una casilla: $i = 0$ es el primero, $i = 1$ el segundo, etc.

$$h : U \times \{0, \ldots, m-1\} \longrightarrow \{0, \ldots, m-1\}.$$

Al variar i, la secuencia $h(k, 0), \ldots, h(k, m-1)$ debe visitar las m casillas sin repetirlas.

- Insertar: avanzar por la secuencia hasta encontrar una casilla disponible.
- Buscar: seguir esa misma secuencia para localizar la clave.
- Si almacenamos n claves, $0 \leq n \leq m$. Cuando $n = m$, la tabla está llena.

La función permanece fija: la misma pareja (k, i) produce la misma casilla. Incrementar i recorre su secuencia; no vuelve a elegir una función al azar.

Sondeo lineal

$h(k, i) = (h_1(k) + i)$ mód m, donde $h_1(k) = k$ mód m es la posición inicial.

Método Sondeo(k, i)

$m \leftarrow |\text{self.casillas}|$
Retornar $((k \text{ mód } m) + i)$ mód m

```
def sondeo(self, k, i):
    m = len(self.casillas)
    return ((k % m) + i) % m
```

Ejemplo: tabla vacía con $m = 9$; insertamos las claves de izquierda a derecha.

Clave	5	28	19	15	20	33
Posiciones probadas	5	1	1, 2	6	2, 3	6, 7
Destino	5	1	2	6	3	7

Después del índice $m - 1$, la siguiente posición es 0; el contador i sigue aumentando. Los bloques de casillas ocupadas pueden crecer y alargar las búsquedas.

Costo: $\Theta(1)$ por sondeo.

El problema de eliminar

$h_1(28) = h_1(19) = 1$. El 19 ocupa la casilla 2 porque la 1 estaba ocupada.

	0	1	2	3	4	5	6	7	8
`T.casillas`	–	28	19	20	–	5	15	33	–

Si eliminamos 28 escribiendo `None`:

	0	1	2	3	4	5	6	7	8
`T.casillas`	–	–	19	20	–	5	15	33	–

Al buscar 19, empezamos por la casilla 1 y encontramos `None`. La búsqueda concluye que 19 no está, aunque permanece en la casilla 2. Una casilla que estuvo ocupada no permite detener la búsqueda como una casilla que nunca se utilizó.

Estados

```
BORRADO = object()
```

Un objeto único distingue una eliminación de una casilla nunca ocupada.

Estado	Al buscar	Al insertar
Clave entera	Comparar con la clave buscada	Seguir sondeando
`None`	Terminar: clave ausente	Ocupar la casilla
`BORRADO`	Continuar la búsqueda	Reutilizar la casilla

Los marcadores se comparan por identidad: `is None`, `is BORRADO`.

Desarrollaremos esta solución para insertar, buscar y eliminar. Las inserciones reciben una clave que todavía no existe en la tabla.

Insertar

Precondición: clave ausente, $m = |\text{self.casillas}| > 0$.

Método InsertarAbierto(k)

$m \leftarrow |\text{self.casillas}|$
Para i desde 0 hasta $m - 1$:
  $q \leftarrow \text{self.Sondeo}(k, i)$
  Si self.casillas[q] es `None`
  o es `BORRADO`:
    self.casillas[q] $\leftarrow k$
    Retornar q
Producir error: tabla llena

```
def insertar_abierto(self, k):
    m = len(self.casillas)
    for i in range(m):
        q = self.sondeo(k, i)
        if (self.casillas[q] is None or
            self.casillas[q] is BORRADO):
            self.casillas[q] = k
            return q
    raise OverflowError("Tabla llena")
```

Ejemplo: `T.insertar_abierto(12)`, tras las inserciones de la slide 17. Prueba la casilla 3 (ocupada por 20) y luego la 4 (vacía). Guarda 12 y devuelve el **índice 4**.

Costo: mejor caso $\Theta(1)$; peor caso $\Theta(m)$, con hasta m sondeos. Si no hay una casilla disponible, produce `OverflowError`.

Buscar

Método BuscarAbierto(k)

$m \leftarrow |\text{self.casillas}|$
Para i desde 0 hasta $m - 1$:
  $q \leftarrow \text{self.Sondeo}(k, i)$
  Si self.casillas[q] es `None`:
    Retornar `None`
  Si self.casillas[q] no es `BORRADO`
  y self.casillas[q] $= k$:
    Retornar q
Retornar `None`

```
def buscar_abierto(self, k):
    m = len(self.casillas)
    for i in range(m):
        q = self.sondeo(k, i)
        if self.casillas[q] is None:
            return None
        if (self.casillas[q] is not BORRADO
                and self.casillas[q] == k):
            return q
    return None
```

Ejemplo: tras insertar 12 en la slide 20, `T.buscar_abierto(12)` prueba las casillas 3 y 4, y devuelve el **índice 4**.

`BORRADO`: continuar. `None`: clave ausente. A lo sumo m sondeos.
Costo: mejor caso $\Theta(1)$; peor caso $\Theta(m)$.

Eliminar

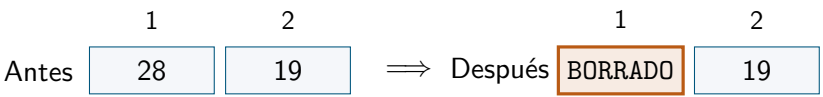
Precondición: `posicion` es el índice de una casilla ocupada por una clave.

Método EliminarAbierto(posicion)

self.casillas[posicion] ← BORRADO

```
def eliminar_abierto(self, posicion):
    self.casillas[posicion] = BORRADO
```

Ejemplo: para eliminar la clave 28:



El índice **0 es válido**: comprobamos `posicion is not None`.
Costo: $\Theta(1)$ con posición conocida.

Doble dispersión

Definición: doble dispersión

$$h(k, i) = (h_1(k) + i\, h_2(k)) \text{ mód } m.$$

- $h_1(k)$: posición inicial.
- $h_2(k)$: tamaño del salto entre intentos; puede depender de la clave.

El sondeo lineal es el caso $h_2(k) = 1$.

Condición: $\gcd(h_2(k), m) = 1$. Son coprimos: su único divisor común positivo es 1. Así visitamos las m casillas antes de repetir una posición.

Contraejemplo: $m = 8$, $h_1(k) = 0$ y $h_2(k) = 2$. Como $\gcd(2, 8) = 2$, repetimos posiciones:

$$0 \longrightarrow 2 \longrightarrow 4 \longrightarrow 6 \longrightarrow 0 \longrightarrow \cdots$$

Las casillas 1, 3, 5 y 7 quedan sin visitar. Si m es primo, cualquier salto entre 1 y $m - 1$ es coprimo con él.

Sondeo doble

Versión alternativa de `sondeo`. Con m primo y $m \geq 3$, obtenemos $1 \leq h_2(k) \leq m-2 < m$.

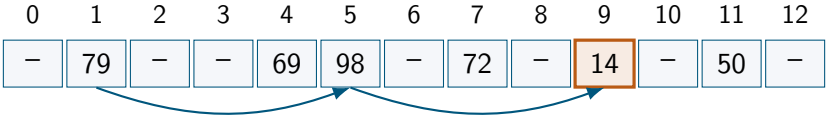
Método Sondeo(k, i)

$m \leftarrow |\text{self.casillas}|$
$h_1 \leftarrow k \bmod m$
$h_2 \leftarrow 1 + (k \bmod (m-2))$
Retornar $(h_1 + i\, h_2) \bmod m$

```
def sondeo(self, k, i):
    m = len(self.casillas)
    h1 = k % m
    h2 = 1 + (k % (m - 2))
    return (h1 + i * h2) % m
```

Ejemplo: $m = 13$, $k = 14$. Obtenemos $h_1(14) = 1$ y $h_2(14) = 4$.

$$h(14,0) = 1, \qquad h(14,1) = 5, \qquad h(14,2) = 9.$$



Las casillas 1 y 5 estaban ocupadas; almacenamos 14 en la casilla 9. **Costo:** $\Theta(1)$ por sondeo.

Cambiar el tamaño

Si aumentamos el tamaño de la tabla, debemos volver a calcular dónde va cada clave.

1. Crear una tabla vacía con el nuevo tamaño.
2. Recorrer los elementos que siguen almacenados.
3. Reinsertarlos usando la nueva función de indexación y su política de colisiones.

Esto permite reducir la carga. Al reconstruir una tabla con direccionamiento abierto, las marcas de borrado no se trasladan.

Ampliar el arreglo sin redistribuir los elementos puede hacer que las búsquedas dejen de encontrarlos. Las implementaciones de esta clase mantienen m fijo.

Diccionarios en Python

`dict` asocia claves con valores de cualquier tipo y gestiona el hash y las colisiones.

Claves de texto

```
notas = {"Ana": 8, "Luis": 9}
notas["Marta"] = 10  # insertar
notas["Ana"] = 9     # actualizar
print(notas["Ana"])  # 9
print("Luis" in notas)  # True
del notas["Luis"]
print(notas.get("Luis"))  # None
```

Claves de tipo `tuple`

```
A = {(0, 0): 2, (1, 2): -3}
print(A[(1, 2)])  # -3
A[(2, 1)] = 5
```

El par (i, j) identifica una entrada de una matriz.

`dict` conserva el orden de inserción; las claves no quedan ordenadas por valor.

Claves hashables

El hash no cambia durante la vida del objeto; claves iguales tienen el mismo hash.

Modificar el objeto no debe alterar su hash ni su comparación por igualdad.

`str` e `int`: sí. `list`, `dict` y `set`: no.

Tuplas: `(1, 2)` sí; `(1, [2, 3])` no.

Comparación

n: claves almacenadas; m: casillas, $n \geq 1$. **Peor caso**, salvo `dict`: **promedio**.

	Directo	**Encadenamiento**	**Abierto**	`dict`
Colisiones	No se producen	Listas dobles	Sondeo lineal o doble	Gestión interna
Buscar	$\Theta(1)$	$\Theta(\ell)$	$\Theta(m)$	$O(1)$
Insertar	$\Theta(1)$	$\Theta(1)$	$\Theta(m)$	$O(1)$
Eliminar (localizado)	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Por clave: $O(1)$
Cuándo conviene	Universo pequeño de claves enteras.	Admitir $n > m$; eliminar por referencia.	Sin nodos; baja ocupación.	Uso general; claves hashables.

`dict` (CPython): buena dispersión y hash/igualdad de costo constante; inserción amortizada. No expone nodos ni casillas.